

CardioReport

Implementation Guide

Complete technical reference for generating deterministic, one page clinical intelligence trend reports from hourly vital sign data. Maps directly to the existing CardioReport module architecture (FastAPI + Pandas + Matplotlib + ReportLab).

Contents

1. Architecture Overview: Why One Page Reports
2. Module Mapping: Your App to This Pipeline
3. `excel_ingest.py`: Data Ingestion (Stage 1)
4. `signal_engine.py`: Statistical Computation (Stage 2)
5. `episodes.py`: Episode Detection (Stage 3)
6. Triage, Trend, and Narrative Logic (Stage 4)
7. `charts.py`: Visualization (Stage 5)
8. `pdf_render.py`: One Page Layout (Stage 6)
9. FastAPI Integration: The `/generate` Endpoint
10. Configuration Reference

1. Architecture Overview: Why One Page Reports

The core problem you described is that your current reports are too long. Here is why one page matters and how the pipeline enforces it.

Why one page?

- A cardiologist spends 30 to 60 seconds on an RPM report. Anything beyond one page gets skimmed or skipped.
- A nurse on shift handoff needs the answer to four questions: Is this patient stable? Are events isolated or patterned? Do HR and RR move together? What should I check?
- Payers evaluate whether RPM adds clinical value. A concise, structured report demonstrates intelligence, not just monitoring.

How the pipeline enforces one page

- The narrative is limited to 6 to 8 sentences by the phrase taxonomy (or by AI prompt constraints).
- The events table is capped at 8 rows maximum, sorted by severity (not time), so only the most important events appear.
- The stats table is fixed at 6 rows (HR avg/min/max, RR avg/min/max) with 5 columns each.
- Charts are sized to fixed pixel dimensions that fit the remaining vertical space.
- Action bullets are capped at 3 to 5 items.

The six stage pipeline

Stage	Your Module	What It Does	Deterministic?
1. Ingest	excel_ingest.py	Read Excel, clean columns, handle missing data	Yes
2. Compute	signal_engine.py	Summary stats, daily aggregates, data quality	Yes
3. Detect	episodes.py	Threshold scanning, episode grouping, severity scoring	Yes
4. Narrate	narrative_ai.py	Phrase taxonomy OR AI narrative + triage + trend	Rules: Yes. AI: No*
5. Chart	charts.py	Candlestick + histogram generation	Yes
6. Render	pdf_render.py	Assemble one page PDF with fixed layout	Yes

* AI narratives vary between runs (different wording, same meaning). All numbers, scores, and triage decisions are always deterministic regardless of AI state.

2. Module Mapping: Your App to This Pipeline

Here is exactly how the report values map to your existing modules. Each section of the report has a clear owner.

Report Element	Source Module	Function / Logic	Can AI Change It?
Triage badge (Green/Yellow/Red)	episodes.py	compute_triage(episodes)	NEVER
Trend assessment label	episodes.py	compute_trend(episodes)	NEVER
Action posture label	episodes.py	compute_action_posture(episodes)	NEVER
Severity scores (S0 to S3)	episodes.py	severity_band(score)	NEVER
Events table rows	episodes.py	detect_episodes(data, thresholds)	NEVER
Events table comments	episodes.py	BAND_CMT[band] lookup	NEVER
Stats table (Mean/Min/Max/P5/P95)	signal_engine.py	compute_stats(windowed_data)	NEVER
Data quality (coverage, low conf)	signal_engine.py	compute_quality(windowed_data)	NEVER
Candlestick chart	charts.py	chart_candlestick(daily_agg, episodes)	NEVER
Histogram chart	charts.py	chart_histogram(windowed_data)	NEVER
Narrative paragraph	narrative_ai.py	taxonomy OR llm_generate()	YES (text only)
Suggested action bullets	narrative_ai.py	taxonomy OR llm_generate()	YES (text only)

The safety boundary is clear: 11 out of 13 report elements are computed by deterministic code that never changes between runs. Only the narrative text and action bullets can be AI generated, and even those have a rule based fallback.

3. excel_ingest.py: Data Ingestion

What this module does

Reads the raw Excel file produced by the monitoring device. Each row is one clock hour. The module cleans column names, drops invalid rows, and returns a pandas DataFrame.

Input columns required

Column	Type	Meaning
Date	datetime	Calendar date of the hour
avg_hr	float	Mean heart rate across all sensor samples in that hour
min_hr	float	Lowest single heart rate reading in the hour
max_hr	float	Highest single heart rate reading in the hour
avg_rr	float	Mean respiratory rate across all sensor samples
min_rr	float	Lowest single respiratory rate reading in the hour
max_rr	float	Highest single respiratory rate reading in the hour
cnt	int	Number of raw sensor samples in the hour (confidence metric)

Cleaning rules

- Drop any row where Date is null (NaT). These are typically header/summary rows in the Excel.
- No imputation. Missing vital sign values stay as NaN and are excluded from calculations via dropna().
- Column rename to snake_case for consistency across modules.
- Parse Date column as datetime using pd.to_datetime().

Window filtering

After ingestion, the data is filtered to the requested time window. This is a simple boolean mask: `df[(df['Date'] >= start) & (df['Date'] <= end)]`. The window boundaries come from the API request parameters (start_date, end_date).

```
def load_and_filter(filepath, start, end):
    df = pd.read_excel(filepath)
    df.columns = ['Date', 'Time', 'avg_hr', 'max_hr', 'min_hr',
                  'avg_rr', 'max_rr', 'min_rr', 'cnt']
    df = df.dropna(subset=['Date'])
    df['Date'] = pd.to_datetime(df['Date'])
    return df[(df['Date'] >= start) & (df['Date'] <= end)]
```

4. signal_engine.py: Statistical Computation

This module takes the windowed DataFrame and computes two output structures: the summary statistics table (for Section 3 of the report) and the daily aggregates (for the candlestick chart in Section 4).

4a. Summary statistics

For each of the 6 vital sign columns, compute 5 values:

Metric	Pandas Operation	Clinical Purpose
Mean	series.mean()	Central tendency. "What is typical for this patient?"
Min	series.min()	Absolute floor. "How low did it ever go?"
Max	series.max()	Absolute ceiling. "How high did it ever go?"
P5	series.quantile(0.05)	Lower bound of "usual". Excludes extreme 5% on the low end.
P95	series.quantile(0.95)	Upper bound of "usual". Excludes extreme 5% on the high end.

All values are rounded to 1 decimal place. P5 and P95 use pandas default linear interpolation. This is important: a cardiologist uses P5/P95 to understand the patient's "usual operating range" without being misled by a single outlier hour.

4b. Daily aggregates (for candlestick chart)

Group all hourly rows by calendar date, then compute:

Output Column	Computation	Maps To In Chart
hr_avg	mean of avg_hr	The dot / average line in HR panel
hr_min	min of min_hr	Bottom of the HR candlestick bar
hr_max	max of max_hr	Top of the HR candlestick bar
rr_avg	mean of avg_rr	The dot / average line in RR panel
rr_min	min of min_rr	Bottom of the RR candlestick bar
rr_max	max of max_rr	Top of the RR candlestick bar
hours	count of rows	Data quality (how many hours recorded that day)
low_cnt	count where cnt < 30	Data confidence flag count for that day

Why min of min_hr (not min of avg_hr)? The candlestick bar shows the full daily range. Using min of avg_hr would understate how low the heart rate actually dropped. Similarly, max of max_hr captures the true daily ceiling. The average dot then shows where the patient spent most of their time.

4c. Data quality metrics

These appear in the report footer:

- **Expected hours** = (end_timestamp minus start_timestamp) in hours + 1
- **Recorded hours** = row count in filtered DataFrame
- **Coverage %** = recorded / expected * 100, rounded to 1 decimal
- **Low confidence hours** = count of rows where cnt < 30

5. episodes.py: Episode Detection

This is the most critical module. It turns raw numbers into clinically meaningful events. Everything downstream (triage, trend, narrative, chart highlighting) depends on the episodes this module produces.

5a. Threshold configuration

Condition	Plain Language	Rule	Column Checked
Bradycardia	Low Heart Rate	avg_hr < 50	avg_hr
Severe bradycardia	Very Low Heart Rate	min_hr < 45	min_hr
Tachycardia	High Heart Rate	avg_hr > 100	avg_hr
Tachypnea	Elevated Breathing Rate	avg_rr > 24	avg_rr
Low confidence	Low Confidence Data	cnt < 30	cnt

Important distinction: Bradycardia checks avg_hr (the hourly average was below 50). Severe bradycardia checks min_hr (at some point during the hour, HR dropped below 45). This means one hour can trigger both conditions simultaneously.

5b. Episode grouping algorithm

The detection algorithm works in four steps:

Step 1: Flag individual hours

For each condition, scan every hour in the window and check the threshold. Mark each hour as flagged=True or flagged=False for that condition.

Step 2: Group consecutive flagged hours into episodes

Walk through the flagged hours in time order. Consecutive flagged hours (adjacent clock hours) belong to the same episode. A gap of 1 or more unflagged hours starts a new episode.

Step 3: Compute episode vitals snapshot

For each episode, compute these values across all hours in the episode:

- **hr_avg_mean** = mean of avg_hr across episode hours
- **hr_min_min** = minimum of min_hr across episode hours (the absolute lowest HR)
- **hr_max_max** = maximum of max_hr across episode hours
- **rr_avg_mean** = mean of avg_rr across episode hours
- **rr_max_max** = maximum of max_rr across episode hours

Step 4: Compute co occurrence flag

Check whether any hour in the episode also meets an opposite channel threshold. Specifically, for a bradycardia or severe bradycardia episode: does any hour also have avg_rr > 24 (tachypnea)? If yes, set cooccur_brady_tachypnea = 1. This is the most clinically significant signal in the entire system.

5c. Severity scoring formula

Each episode gets a severity_score computed as:

Component	Value	Rationale
Base (Severe brady)	+5	Highest base because HR < 45 is clinically urgent
Base (Bradycardia)	+3	Concerning but may be medication related
Base (Tachycardia)	+3	Concerning, especially if sustained
Base (Tachypnea)	+2	Lower base because RR is noisier than HR
Duration bonus	+1 per hour beyond first	Longer episodes are more concerning than brief ones

Coupling bonus	+2 if co occurrence	Concurrent HR + RR abnormality elevates concern
Low confidence penalty	1 if any low cnt hour	Reduce score if data quality is questionable

5d. Severity bands

Band	Score	Row Color	Comment Phrase
S0	0 to 4	No highlight	Brief deviation; continue monitoring
S1	5 to 8	Amber	Sustained deviation; review context
S2	9 to 12	Red	Sustained pattern; consider provider review
S3	13+	Red	Critical sustained pattern; urgent review advised

6. Triage, Trend, and Narrative Logic

These three outputs are computed from the episodes list. All three use simple if/else chains. They are 100% deterministic.

6a. Triage color (header badge)

Rule (checked in order)	Result
Any severe brady episode ≥ 4 hours	RED
Any tachypnea episode ≥ 8 hours	RED
Any coupled episode AND max severity ≥ 9	RED
Max severity score ≥ 5 (but no RED trigger above)	YELLOW
Everything else	GREEN

6b. Trend assessment (one of four labels)

Rule	Label
Max severity ≥ 9 OR (coupled AND total episode hours > 10)	Progressively unstable vital sign pattern
Max severity ≥ 5 OR total episode hours > 5	Intermittently unstable vital sign pattern
No episodes or all below thresholds	Stable vital sign pattern

6c. Phrase taxonomy narrative (USE_LLM = false)

When AI is disabled, the narrative is assembled from 5 components, each selected by a fixed rule:

Component	Selection Rule	Example Output
1. Burden	Count of episodes + type set	"8 episodic events detected, including: Very Low Heart Rate, Elevated Breathing Rate..."
2. Distribution	0 to 1 eps = isolated 2 to 3 = intermittent 4+ = clustered	"Events occurred in clustered bursts rather than as isolated occurrences."
3. Duration	Any episode ≥ 2 hours?	"Several episodes persisted for multiple consecutive hours (longest: 9 hours)..."
4. Coupling	Any cooccur flag = 1?	"Periods of low heart rate coincided with elevated breathing rate..."
5. Data quality	Low conf hours > 3 ?	"Data confidence was reduced during N hours; interpret cautiously."

Each component either includes its sentence or omits it entirely. The components are concatenated in order to form the paragraph. Same episodes in, same paragraph out, every time.

6d. Suggested clinical review actions (USE_LLM = false)

Action bullets are selected based on which episode types exist in the window:

If Detected	Action Bullet Included
Bradycardia or Severe bradycardia	"If sustained or recurrent low heart rate... review symptoms, blood pressure, medications..."
Co occurrence (coupled HR/RR)	"If low heart rate accompanies elevated breathing rate... assess for compensatory stress..."
Tachycardia	"If prolonged high heart rate... evaluate for pain, infection, dehydration..."
Tachypnea	"If elevated breathing rate persists... correlate with oxygen saturation..."
Any episodes at all	"If event frequency or duration is increasing... consider earlier provider notification..."

7. charts.py: Visualization

Chart A: Daily Candlestick (full width, HR top, RR bottom)

This is the primary visual. It compresses hourly noise into a daily summary that a cardiologist can scan in seconds.

How each day is drawn:

- **Vertical bar:** Drawn from hr_min (bottom) to hr_max (top) for HR panel. Same for RR panel with rr_min and rr_max.
- **Dot on bar:** Placed at hr_avg (or rr_avg). Connected across days with a thin line to show baseline drift.
- **Bar color:** RED if that calendar day contains any hour that belongs to a detected episode. BLUE (HR) or ORANGE (RR) otherwise.
- **Reference lines:** Horizontal dashed lines at 50 bpm and 100 bpm (HR panel), 24 brpm (RR panel).

Episode day detection for chart coloring:

For each episode, expand its start and end timestamps to full calendar days. Build a set of all episode dates. When drawing each daily bar, check if that date is in the set. If yes, color red. If no, color blue/orange.

Chart dimensions:

figsize=(7.2, 3.0) inches at 200 DPI. Two subplots stacked vertically with height ratio 1:0.8 and 0.08 inch horizontal padding. This fits the available width on a letter size page with 0.5 inch margins.

Chart B: Histogram (HR left, RR right)

Shows the distribution of all hourly average values across the window.

What the histogram reveals that the candlestick cannot:

- Is the distribution normal or skewed? A left tail in HR means the patient is spending significant time at low heart rates.
- Is the distribution narrow or wide? A wide spread indicates high variability (instability).
- Are there bimodal peaks? Two clusters might indicate distinct physiologic states (e.g., medication on vs off).

Histogram construction:

- Data: avg_hr column (all hourly averages in the window). 25 bins. Same for avg_rr with 20 bins.
- Dashed vertical line at the mean value.
- Dotted vertical lines at threshold values (50 and 100 for HR, 24 for RR).
- figsize=(7.2, 1.7) at 200 DPI. Side by side layout with 3 units of horizontal padding.

8. pdf_renderer.py: One Page Layout

The layout engine assembles all computed outputs into a fixed structure using ReportLab. The key constraint is: **everything must fit on one page**. Here is how each section is sized to guarantee this.

Section	Approx Height	Key Constraint
Header + meta + disclaimer	~0.9 inch	Fixed height. Title bar + 1 row meta + 1 line disclaimer.
Section 1: Narrative + actions	~1.8 inch	Narrative: 6 to 8 sentences at 8.5pt = ~1 inch. Trend + action rows: ~0.3 inch. Actions: 3 to 5 bullets at 8pt = ~0.5 inch.
Section 2: Events table	~1.2 inch	Header + max 8 rows at ~0.15 inch per row.
Section 3: Stats table	~1.0 inch	Header + 6 rows. Fixed height always.
Section 4: Candlestick chart	~2.5 inch	Image rendered at fixed figsize.
Section 4: Histogram chart	~1.5 inch	Image rendered at fixed figsize.
Footer	~0.3 inch	Single line of small text.
TOTAL	~9.2 inch	Letter page usable height: 10.2 inch (11 inch minus 0.4+0.4 margins)

The math works: 9.2 inches of content fits within 10.2 inches of usable page height, leaving about 1 inch of breathing room for spacing between sections. If the narrative runs long (e.g., AI generates 8 full sentences), the buffer absorbs it. If a window has zero episodes, Section 2 shrinks to 1 row ("No events detected"), giving even more room.

Report page settings

- Page size: US Letter (8.5 x 11 inches)
- Top margin: 0.4 inches
- Bottom margin: 0.35 inches
- Left/right margins: 0.5 inches each
- Content width: 7.5 inches
- Font: Helvetica family (built into ReportLab, no external dependencies)
- Body text: 8.5pt with 12pt leading
- Table text: 7.5pt with 10pt leading

9. FastAPI Integration: The /generate Endpoint

Here is how the pipeline plugs into your existing FastAPI server. The endpoint receives a patient ID and date range, runs the six stages, and returns a PDF.

```
# main.py (FastAPI)

@app.post('/api/generate-report')
async def generate_report(req: ReportRequest):
    # Stage 1: Ingest
    df = excel_ingest.load_patient(req.patient_id)

    # Stage 2: Window + Compute
    wdf = df[(df['Date'] >= req.start) & (df['Date'] <= req.end)]
    stats = signal_engine.compute_stats(wdf)
    daily = signal_engine.compute_daily(wdf)
    quality = signal_engine.compute_quality(wdf, req.start, req.end)

    # Stage 3: Detect
    episodes = episodes_mod.detect_episodes(wdf, THRESHOLDS)

    # Stage 4: Narrate
    triage_color = episodes_mod.compute_triage(episodes)
    trend_label = episodes_mod.compute_trend(episodes)
    action_posture = episodes_mod.compute_action_posture(episodes)

    if USE_LLM:
        try:
            narrative, actions = narrative_ai.generate(
                stats, episodes, quality, triage_color, trend_label)
        except Exception:
            narrative = narrative_ai.taxonomy_fallback(
                wdf, episodes, req.period_label)
            actions = narrative_ai.actions_fallback(episodes)
        else:
            narrative = narrative_ai.taxonomy_fallback(
                wdf, episodes, req.period_label)
            actions = narrative_ai.actions_fallback(episodes)

    # Stage 5: Chart
    candlestick_buf = charts.candlestick(daily, episodes)
    histogram_buf = charts.histogram(wdf)

    # Stage 6: Render
    pdf_bytes = pdf_render.build_one_page(
        patient_id=req.patient_id,
        period_label=req.period_label,
        triage=triage_color,
        narrative=narrative,
        trend=trend_label,
        action_posture=action_posture,
        actions=actions,
        episodes=episodes,
        stats=stats,
        quality=quality,
        candlestick_img=candlestick_buf,
        histogram_img=histogram_buf,
```

```

)

return StreamingResponse(
    io.BytesIO(pdf_bytes),
    media_type='application/pdf',
    headers={'Content-Disposition':
        f'attachment; filename=report_{req.patient_id}.pdf'})

```

Critical architecture point: The triage color, trend label, and action posture are computed BEFORE the narrative is generated. They are passed INTO the AI prompt as pre computed facts. The AI cannot override them. This is the safety boundary.

10. Configuration Reference

Parameter	Default	Affects	Module
brady_hr_avg	50 bpm	Low Heart Rate episode detection	episodes.py
severe_brady_hr_min	45 bpm	Very Low Heart Rate episode detection	episodes.py
tachy_hr_avg	100 bpm	High Heart Rate episode detection	episodes.py
tachy_rr_avg	24 brpm	Elevated Breathing Rate detection	episodes.py
low_cnt_threshold	30	Low confidence data flagging	signal_engine.py
USE_LLM	false	Narrative source (AI vs taxonomy)	narrative_ai.py
max_events_table	8	Maximum rows in events table	pdf_render.py
max_actions	5	Maximum suggested action bullets	narrative_ai.py
chart_dpi	200	Chart resolution	charts.py

Changing any threshold recascades the entire pipeline. If you change brady_hr_avg from 50 to 48, fewer hours get flagged, episodes change, severity scores change, triage may change, narrative phrases change, chart highlighting changes. The pipeline is designed so that every downstream output automatically reflects threshold changes with zero manual intervention.